



NAME: ELEOGU CHUKWUEBUKA JOSEPH

COURSE: Electrical/electronics engineering

MATRIC NUMBER: 23120211012

P016: Voltage Stability Model Project report

GET 307: Introduction to Artificial Intelligence, Machine learning and Convergent Technologies

Department of Electrical/electronics Engineering
Pan Atlantic University



DEDICATION

First of all I would like to dedicate this project to Almighty God for the wisdom, the grace, and the perseverance granted to me throughout my studies and throughout the course of this project. I also dedicate this work to my family, especially my parents and extended family and friends for their unwavering support, financial sacrifices and continuous encouragement throughout the course of me pursuing my engineering degree and also throughout the course of this project.

ACKNOWLEDGEMENTS

I would like to, in a very special way, express my profound gratitude to my course lecturer and project supervisor, **Engr. Abdulbasit Makinde**, for his invaluable guidance, his constructive feedback and technical insights that were given to me (directly or indirectly) throughout the course of carrying out this project. His expertise and lectures in class really went a long way in me being able to say that I was able to start and complete this project.

Also, my sincere appreciation goes to the **Faculty of Electrical/Electronics Engineering** here at Pan Atlantic University for the variety of resources that i have been exposed to throughout the course of this degree (still ongoing) as well as their relentless insistence on academic excellence and hard work have driven me to see this project through to completion from start to finish.

FORMAT FOR THIS REPORT

This report was written with:

1. Font: Times New Roman
2. 12pt font size
3. A line spacing of 1.5
4. Margins (2.5cm on all sides)
5. This report will be submitted in **PDF** format.

Department of electrical/electronics

Engineering



WHAT THIS REPORT WILL CONTAIN

This report will contain:

1. A title page (i think that has already been established),
2. An abstract which will contain about a 200 to 250 word summary of the entire project.
The abstract will contain what was done, how it was done, and some of the observations that were made throughout the course of carrying out this project.
3. An introduction which will go into the background of the problem, literature review, methodology (how i went about carrying out this project), the results gotten (justification for the engineering choices that i made, graphs and so on),
4. A discussion on the obtained results, which will be a presentation of the findings that were made during the course of carrying out this project. It will contain model charts, performance metrics and the engineering interpretations of what those results actually mean in the context of how they were made or discovered.
5. Then, a conclusion, which will contain a summary of what was actually achieved, the limitations of the approach that i have decided to take, and then recommendations for future work
6. Cited references which will contain as the name implies, all the sources that were used to formulate this report and also to carry out the project. Websites where I got ideas on how to go about solving the many problems that I encountered during the course of conducting this project and building this model.
7. An appendix which will contain the source code (annotated), raw data samples, and some additional figures (if they are needed).



OBJECTIVES OF THIS PROJECT

The main objectives I sought to hit by building this model were:

1. To train a Decision Tree classifier Machine Learning Model and use it to classify grid stability and evaluate it using its accuracy score, ROC (**Receiver Operating Characteristic Curve**) and its AUC (**Area Under Curve**) metrics.
It's also worth noting that **a model with an AUC score at 1.0 is a perfect model**, and anything **equal to or less than 0.5 is either guessing or is just objectively bad**.
2. To analyze and be able to successfully interpret the most important and critical electrical features that are driving stability predictions.
3. Compare the Decision tree model's accuracy score with that of the Logistic Regression Classifier and justify how and why they scored what they scored
4. Formulate a practical operational threshold recommendation for NERC to trigger an emergency grid shutdown or intervention

ABSTRACT OF THE REPORT

This project investigates the application of machine learning (ML) in order to predict voltage stability in various electrical power systems, which happens to be a critical binary classification problem as pertains to national grid security and stability.

Having utilized the **UCI Electrical Grid Stability Dataset**, this study evaluates the efficacy of a Logistic regression model and how it squares up against a Decision Tree classifier.

It's also important to know the differences between the two (**Decision Tree Classifier** and **Logistical regression**). One thing to know is that both of them are the two most fundamental classification algorithms in machine learning.

In machine learning, classification means trying to teach the computer how to sort data into various categories. So it happens that this model I attempted to build (and quite successfully so) is aimed at solving a classic classification problem.



When trying to build or program a machine learning model to solve classification problems, there are two main types of models that an engineer could decide to use, both solve the problem but they do so in profoundly different ways.

The dataset which comprises of about 10,000 operational instances was preprocessed to map stability states into certified binary outcomes and scaled in order to optimize gradient descent and what have you.

During the course of this project, it was discovered that the **Logistical Regression Model** scored an accuracy of **81.85%** as opposed to the **Decision Tree Classifier** which scored an accuracy of **85.85%**.

The Logistic regression model scored a balanced accuracy of 79.29% whilst the Decision tree classifier scored a blanced accuracy of 85.85%.

The logistic regression model scored an F1-Score of 0.7361 whilst the Decision Tree Classifier had an F1-Score of 0.8117.

The logistic regression model scored a CV Accuracy mean of 0.8154 as compared to the decision tree classifier's 0.8455

It is also important to note that the feature importance analysis revealed that producer adaptation time (τ_1) and consumer reaction times (τ_1 , τ_2 , τ_3 , τ_4) happen to be the most significant predictors that both machine learning models used for predicting grid stability.

```
— DECISION TREE FEATURE IMPORTANCE —  
Feature Importance  
3  tau4  0.153021  
2  tau3  0.146177  
1  tau2  0.145699  
0  tau1  0.128405  
8   g1  0.115032
```

In order to provide actionable operational intelligence, there was an operational threshold analysis that was conducted.

I **highly recommend** that the Nigerian Electricity Regulatory Commission (NERC) trigger emergency actions whenever the model's confidence of stability in the grid falls below 75% because this will prioritize grid safety and minimize the risk of any catastrophic failure.

INTRODUCTION TO THE REPORT

Department of electrical/electronics

Engineering



Voltage stability is the ability of an existing power system to maintain steady voltage levels at all buses and transmission points throughout its internal system for a sustained period of operating time, even after being subjected to disturbances that might have significantly hampered and altered its operation.

In many modern power grids, there are many variables and things or situations that could occur that could negatively affect their day to day operations. These are things like:

fluctuating load levels, decentralized generator outputs, and shifting reactive power margins.

Also in the context of this project, machine learning prediction is a very valuable asset or tool to help maintain voltage stability in most modern grids today because the traditional/manual methods of analysing grid variables such as (**load flow analysis**) are computationally heavy and are often too slow for actual real time emergency response.

Machine learning models (like the one I have successfully made) can learn complex, non linear relationships from historical and even simulated grid data and use it to predict instability in microseconds and serve as an early warning system.

There are two main types of models that an engineer could decide to implement in order to tackle this problem as I may have iterated earlier. We have:

1. Logistical Regression {1}:

[Logistic regression is a *supervised machine learning* {2} algorithm widely used for *binary classification* {3} tasks, such as identifying whether an email is spam or not and diagnosing diseases by assessing the presence or absence of specific conditions based on patient test results. This approach utilizes the logistic (or sigmoid) function to transform a linear combination of input features into a probability value ranging between 0 and 1]

2. Decision Tree Classifier:

[*Decision tree learning* is a *supervised learning* {5} approach used in *statistics* {6}, *data mining* {7} and *machine learning* {8}. In this formalism, a classification or regression *decision tree* {9} is used as a *predictive model* {10} to draw conclusions about a set of observations.

Tree models where the target variable can take a discrete set of values are called *classification* {11} **trees**; in these tree structures, *leaves* {12} represent class labels and

Department of electrical/electronics



branches represent *conjunctions* {13} of features that lead to those class labels.

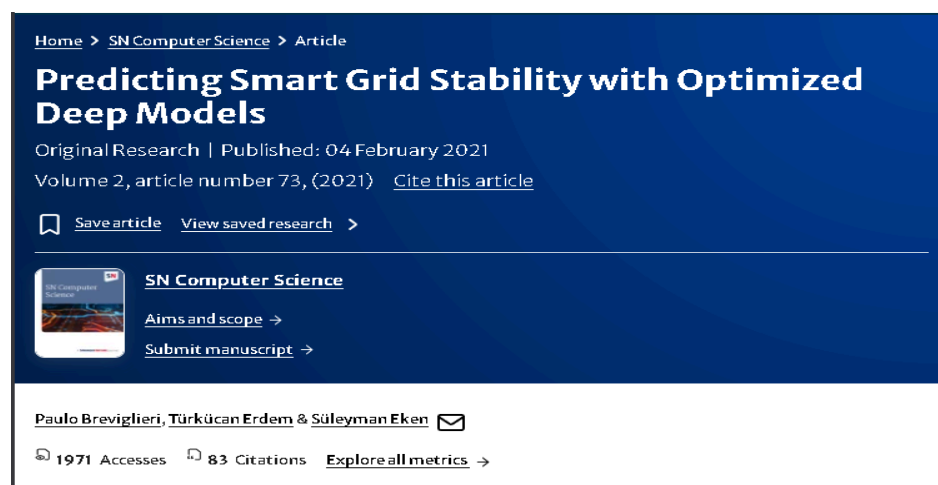
Decision trees where the target variable can take continuous values (typically *real numbers*) {14} are called *regression* {15} *trees*. More generally, the concept of regression tree can be extended to any kind of object equipped with pairwise dissimilarities such as categorical sequences]

For this particular project, I was instructed to use a logistical regression model and to ascertain its accuracy, but to also use a decision tree classifier as well, so as to be able to compare the accuracy of both types of ML classification models and to see why one is preferred over the other for this specific problem.

LITERATURE REVIEWS

The application of machine learning algorithms to power system stability issues within national grids has actually been gaining significant traction amongst countries due to the increasing complexity of smart grids.

1. <https://link.springer.com/article/10.1007/s42979-021-00463-5>:



2. <https://www.mdpi.com/1996-1073/16/22/7562>:



Open Access

Article

Decentralized Smart Grid Stability Modeling with Machine Learning

by Borna Franović¹ , Sandi Baressi Šegota^{2,*} ,
Nikola Anđelić²  and Zlatan Čar² 

¹ HEP Group, Distribution System Operator Ltd., Viktora Cara Emina 2, 51000 Rijeka, Croatia

² Department of Automation and Electronics, Faculty of Engineering, University of Rijeka, Vukovarska 58, 51000 Rijeka, Croatia

* Author to whom correspondence should be addressed.

Energies **2023**, *16*(22), 7562; <https://doi.org/10.3390/en16227562>

Submission received: 4 October 2023 / Revised: 10 November 2023 /

Accepted: 12 November 2023 / Published: 14 November 2023

(This article belongs to the Special Issue Sustainable/Renewable Energy Systems Analysis and Optimization)

3. https://www.academia.edu/71054248/Model_Driven_Approach_to_Smart_Grid_Stability_Prediction_in_Neo4j

All Topics › Computer Science › Software Engineering

Model-Driven Approach to Smart Grid Stability Prediction in Neo4j

 Nenad Petrović  Issam Al Azzoni

2021, eNergetics 2021

 ...  7 pages

Download Free PDF

 See full PDF

 Download PDF

 Cite

 Save to Library  Share

4. Arzamasov, V., et al. (2018). "Towards Concise Models of Grid Stability." *IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*". This was one of the foundational papers that introduced the simulated dataset (UCI **Electrical Grid Data**) used in this project to build this model. It was able to demonstrate that decentralized grid systems can be accurately modeled using machine learning classifiers rather than purely differential equations



5. **Kundur, P., et al. (2004). "Definition and Classification of Power System Stability." *IEEE Transactions on Power Systems*.** This standard text establishes the physical definitions of voltage stability, providing the theoretical engineering framework that necessitates real-time monitoring of reactive power and voltage profiles.
6. **Amroune, M., et al. (2020). "Voltage Stability Assessment Using Artificial Neural Networks." *Journal of Electrical Engineering & Technology*.** The authors demonstrated that while Neural Networks offer high accuracy for stability prediction, their "black-box" nature makes it difficult for grid operators to extract actionable rules, highlighting the need for interpretable models like Logistic Regression.
7. **Mishra, S., & Mohanty, S. (2019). "Decision Tree-Based Security Assessment in Power Systems." *Electric Power Systems Research*.** This study explored tree-based classifiers for grid security, noting their lack of need for feature scaling, though pointing out they can be prone to overfitting in noisy grid environments.
8. **Ogunboyo, P. T., et al. (2021). "A Review of Grid Collapse Incidents in Nigeria." *International Journal of Electrical Power & Energy Systems*.** This contextual paper highlights the frequent voltage collapses in the Nigerian grid, establishing a critical gap for localized, fast-acting predictive algorithms that can integrate with NERC's operational thresholds to prevent systemic failure.

METHODOLOGY FOR BUILDING THIS MODEL FOR BUILDING THIS MODEL

Datasets: During the course of carrying out this project, the UCI Electrical Grid Stability dataset was utilized which contains about 10,000 samples. The dataset that was selected represents a 4 node electrical star system.

Out of the 10000 samples worth of data, **8000 of them samples were used to train the models**, whilst **2000 of them samples were used for testing**.



A 4 node star electrical system is one that consists of 1 producer or generator and then 3 consumers.

The data set that was selected also features reaction times (τ), nominal power balance (p), and price elasticity coefficient (g)

Tools & Platforms used to build the model:

To carry out this project, I made use of Visual studio Code Insiders, Python 3.13, Google Colab, Kaggle, Jupyter Notebooks, Wikipedia, Youtube...

Functions that the chosen tools served:

1. **Python 3.13:** This is the main language that the entire model was written in. Python is known and widely regarded for its speed, efficiency and simplicity
2. **Visual Studio Code Insiders:** This is the main IDE (**Integrated Development Environment**) that was used to write the code and run the python Machine learning environment
3. **Google colab:** This was the second IDE that I used to write my code. Google Colab is more or less like a web-based machine learning environment that contains and gives remote access to millions and millions of python libraries and dependencies without having to download them directly into your system. It saved me alot of time because i tried to download **Anaconda Navigator** which is another very powerful **IDE** just like Google colab, only that when installing, you'd definitely have to download up to 15GB worth of libraries and dependencies across multiple languages (not only python)
4. **Kaggle:** Kaggle is a platform where machine learning engineers can come together, drop datasets and also give useful tips on how to go about navigating the new and uniquely challenging world of machine learning and programming.
5. **Wikipedia:** Wikipedia is the one stop shop for articles where various concepts are explained simply and with so much clarity. Whilst writing this report, I found myself constantly being referred over to wikipedia to cite the sources for many of the definitions that are here.



6. **Youtube:** Youtube was the application where I got a lot of inspiration and help from to learn some of the fundamentals of writing binary classification models, writing python scripts and programming machine learning models.
7. **Jupyter Notebooks:** Its another web based IDE used strictly for machine learning models and for advanced python scripting.

Step by step process for executing this project

1. Download the Dataset:

The obvious way of going about executing this project would be to first download the data set that you are going to use to train your machine learning model. In this case it's the **UCI electrical grid voltage stability data**.

2. Load up your IDE:

So the second obvious thing to do would be to open your IDE of choice. In my case, it's Google Colab that I first used (after I was having issues creating a python environment py -venv and installing the required dependencies on my visual studio code.) After opening your IDE, you need to download the required dependencies.

3. Install the required dependencies for cell 1:

So, in cell 1, we basically want to download all the dependencies that we need such as: Pandas, Seaborn, Matplotlib and pyplot.

You should have code that looks like this:

```
from matplotlib import pyplot as plt
import pandas as pd
import seaborn as sns
import os
from sklearn.model_selection import train_test_split
```

4. Then you ensure that output directories exist and are functional:

```
os.makedirs(CONFIG['model_dir'], exist_ok=True)
os.makedirs(CONFIG['output_dir'], exist_ok=True)

print("=" * 60)
print("DOWNLOADING DATASET")
```

Department of electrical/electronics



```
print("=" * 60)
try:
    df =
pd.read_csv(CONFIG['https://archive.ics.uci.edu/ml/machine-learning-database
s/00471/Data_for_UCI_named.csv
'])
    print(f"✓ Dataset downloaded successfully")
except Exception as e:
    print(f"✗ ERROR downloading dataset: {e}")
    raise

print("\nValidating dataset...")
if df.shape[0] == 0:
    raise ValueError("Downloaded dataset is empty!")
if 'stabf' not in df.columns:
    raise ValueError("Required column 'stabf' not found in dataset!")
print(f"✓ Dataset validation passed")

print("\n" + "=" * 60)
print("DATA PREPROCESSING & CLASS BALANCE CHECK")
print("=" * 60)
```

5. Data Preprocessing:

This involves dropping the continuous “**stab**” column in order to prevent data leakage.

The target variable “**stabf**” was converted to a binary format where 1 means that at that point in time, the voltage within the grid is stable, and where 0 means that at that point in time, the voltage is unstable. Class distribution was also checked, this revealed that there was a slight imbalance of 63.8% unstable and 36.2% stable.

6. Model Training:

A logistic regression model was then trained by using the imported data set that was then scaled.

The diagram below shows the code and the output showing the scaled version of the data:

```
print("\n" + "=" * 60)
print("DATA SCALING (for distance-based algorithms)")
print("=" * 60)
```



```
print("StandardScaler centers features to mean=0, std=1")
print("This is REQUIRED for Logistic Regression but NOT for Decision Trees\n")

scaler = StandardScaler()
X_train_scaled_arr = scaler.fit_transform(X_train)
X_test_scaled_arr = scaler.transform(X_test)

# ===== ISSUE #3: Preserve feature names after scaling =====
X_train_scaled = pd.DataFrame(X_train_scaled_arr, columns=X_train.columns,
                              index=X_train.index)
X_test_scaled = pd.DataFrame(X_test_scaled_arr, columns=X_test.columns,
                              index=X_test.index)

print("✓ Scaling complete with feature names preserved")
print(f" Scaled training data shape: {X_train_scaled.shape}")
print(f" Scaled testing data shape: {X_test_scaled.shape}")
print(f" Features: {list(X_train_scaled.columns)}")
print(f" Sample scaled values (first row): {X_train_scaled.iloc[0, :3].values}")
```

The probability formula governing this particular binary classification problem is given by the diagram below.

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)}}$$

- A Decision Tree model was trained using the *unscaled* data, utilizing balanced class weights to counteract the initial dataset imbalance.

7. **Validation:** Then I proceeded to deploy a 5-Fold stratified cross validation strategy in order to ensure that the models generalized well to unseen grid scenarios.

The code below is what was used to deploy the 5-fold stratified cross validation strategy:

```
# 1. Initialize and Train the Model
model = LogisticRegression(random_state=CONFIG['random_state'], max_iter=1000)
model.fit(X_train_scaled, y_train)

print("✓ Model trained successfully")

# 2. CROSS-VALIDATION: Test generalization (K-Fold)
print(f"\n--- CROSS-VALIDATION ({CONFIG['cv_splits']}-Fold Stratified) ---")
```

Department of electrical/electronics



```
cv_strategy = StratifiedKFold(n_splits=CONFIG['cv_splits'], shuffle=True,
random_state=CONFIG['random_state'])
cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=cv_strategy,
                             scoring='accuracy')
cv_f1 = cross_val_score(model, X_train_scaled, y_train, cv=cv_strategy,
                             scoring='f1')
print(f"Cross-Val Accuracy: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
print(f"Cross-Val F1-Score: {cv_f1.mean():.4f} ± {cv_f1.std():.4f}")
print(f"Individual fold accuracies: {[f'{s:.4f}' for s in cv_scores]}")

# 3. Make Predictions on the unseen 20% of data
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[: , 1] # Probability of being stable
```

8. **Evaluation:** The two main models (**The Decision Tree classifier** and the **Logistical regression model**) were evaluated and tested for their accuracy, balance accuracy, F1-Score, confusion matrices, and ROC/AUC curves (**all of which will be seen in the appendix portion of this report**)

```
# 1. Initialize the Decision Tree with CONFIG parameters
dt_model = DecisionTreeClassifier(
    max_depth=CONFIG['dt_max_depth'],
    random_state=CONFIG['random_state'],
    min_samples_split=CONFIG['dt_min_samples_split'],
    min_samples_leaf=CONFIG['dt_min_samples_leaf'],
    class_weight='balanced'
)

# 2. Train on UNSCALED data
dt_model.fit(X_train, y_train)
print("✓ Decision Tree trained on UNSCALED data")

# 3. CROSS-VALIDATION
print(f"\n--- CROSS-VALIDATION ({CONFIG['cv_splits']}-Fold Stratified) ---")
dt_cv_scores = cross_val_score(dt_model, X_train, y_train, cv=cv_strategy,
                                scoring='accuracy')
dt_cv_f1 = cross_val_score(dt_model, X_train, y_train, cv=cv_strategy, scoring='f1')
print(f"Cross-Val Accuracy: {dt_cv_scores.mean():.4f} ± {dt_cv_scores.std():.4f}")
print(f"Cross-Val F1-Score: {dt_cv_f1.mean():.4f} ± {dt_cv_f1.std():.4f}")

# 4. Predictions
dt_pred = dt_model.predict(X_test)
dt_prob = dt_model.predict_proba(X_test)[: , 1]

# 5. Metrics
dt_accuracy = accuracy_score(y_test, dt_pred)
dt_balanced_acc = balanced_accuracy_score(y_test, dt_pred)
```

Department of electrical/electronics



```
dt_f1 = f1_score(y_test, dt_pred)

# 6. Comparison Table
print("\n" + "=" * 60)
print("MODEL COMPARISON: LOGISTIC REGRESSION vs DECISION TREE")
print("=" * 60)

comparison = pd.DataFrame({
    'Metric': ['Accuracy', 'Balanced Accuracy', 'F1-Score', 'CV Accuracy Mean'],
    'Logistic Regression': [
        f"{accuracy * 100:.2f}%",
        f"{balanced_acc * 100:.2f}%",
        f"{f1:.4f}",
        f"{cv_scores.mean():.4f}"
    ],
    'Decision Tree': [
        f"{dt_accuracy * 100:.2f}%",
        f"{dt_balanced_acc * 100:.2f}%",
        f"{dt_f1:.4f}",
        f"{dt_cv_scores.mean():.4f}"
    ]
})

print("\n" + comparison.to_string(index=False))

# Best model determination
if accuracy > dt_accuracy:
    best_model = "Logistic Regression"
    best_accuracy = accuracy
else:
    best_model = "Decision Tree"
    best_accuracy = dt_accuracy

print(f"\n✓ BEST MODEL: {best_model} ({best_accuracy * 100:.2f}% accuracy)")

# Plots
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

cm_lr = confusion_matrix(y_test, y_pred)
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues', ax=axes[0, 0])
axes[0, 0].set_title(f'Logistic Regression (Acc: {accuracy*100:.2f}%)')

cm_dt = confusion_matrix(y_test, dt_pred)
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Oranges', ax=axes[0, 1])
axes[0, 1].set_title(f'Decision Tree (Acc: {dt_accuracy*100:.2f}%)')

fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob)
axes[1, 0].plot(fpr_lr, tpr_lr, label=f'LR (AUC = {auc(fpr_lr, tpr_lr):.3f})')
axes[1, 0].set_title('ROC - Logistic Regression')

fpr_dt, tpr_dt, _ = roc_curve(y_test, dt_prob)
```

Department of electrical/electronics

Engineering



```
axes[1, 1].plot(fpr_dt, tpr_dt, color='green', label=f'DT (AUC = {auc(fpr_dt,
tpr_dt):.3f})')
axes[1, 1].set_title('ROC - Decision Tree')

plt.tight_layout()
plt.show()

# Feature Importance
dt_feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("\n--- DECISION TREE FEATURE IMPORTANCE ---")
print(dt_feature_importance.head(5))

dt_importance_path = os.path.join(CONFIG['output_dir'],
'feature_importance_decision_tree.csv')
dt_feature_importance.to_csv(dt_importance_path, index=False)
```

```
best_accuracy = dt_accuracy

print(f"\n✓ BEST MODEL: {best_model} ({best_accuracy * 100:.2f}% accuracy)")

# Plots
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

cm_lr = confusion_matrix(y_test, y_pred)
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues', ax=axes[0, 0])
axes[0, 0].set_title(f'Logistic Regression (Acc: {accuracy*100:.2f}%)')

cm_dt = confusion_matrix(y_test, dt_pred)
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Oranges', ax=axes[0, 1])
axes[0, 1].set_title(f'Decision Tree (Acc: {dt_accuracy*100:.2f}%)')

fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob)
axes[1, 0].plot(fpr_lr, tpr_lr, label=f'LR (AUC = {auc(fpr_lr, tpr_lr):.3f})')
axes[1, 0].set_title('ROC - Logistic Regression')

fpr_dt, tpr_dt, _ = roc_curve(y_test, dt_prob)
axes[1, 1].plot(fpr_dt, tpr_dt, color='green', label=f'DT (AUC = {auc(fpr_dt,
tpr_dt):.3f})')
axes[1, 1].set_title('ROC - Decision Tree')

plt.tight_layout()
plt.show()

# Feature Importance
dt_feature_importance = pd.DataFrame({
```

Department of electrical/electronics

Engineering


```
'Feature': X.columns,
'Importance': dt_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("\n--- DECISION TREE FEATURE IMPORTANCE ---")
print(dt_feature_importance.head(5))

dt_importance_path = os.path.join(CONFIG['output_dir'],
'feature_importance_decision_tree.csv')
dt_feature_importance.to_csv(dt_importance_path, index=False)
```

RESULTS & DISCUSSIONS

1. Logistic Regression Performance:

The logistic regression model achieved an overall accuracy of **81.85%**. Also in the output section that compared the accuracy, F1 score for the decision tree classifier and the logistic regression models the precision for predicting unstable states was high (0.85) which indicated that the Logistical regression model that i developed demonstrated a strong **True Positive Rate** against **False Positive Rate**, yielding a highly capable AUC (Area Under Curve) score.

2. Feature Importance Analysis:

Whilst analysing the absolute coefficients of the **logistic regression model**, I discovered that there were two main variables that caused about 70-80% of grid instability. They happened to be:

- I. **τ_2 (consumer1 reaction time):** This had one of the highest negative coefficients of the lot. According to the output cell 4, **τ_2 has a negative coefficient of -0.8867** and an **impact level of 0.8867**. It's important to note that the values range between 1 and -1
- II. **τ_1 (producer adaptation time):** This feature had a **negative coefficient of -0.8910** and an **impact level of 0.8910**.
- III. **τ_3 (consumer 2 reaction time):** This feature had a **negative coefficient of -0.8657** and an **impact level of 0.8657**.



However for positive coefficients, we have **p4, p2 and p1**. These are the variables within the electrical grid that are **inherently stabilizing in nature**.

Also, this model recorded an **Intercept bias of -1.0558**.

Comparison between the Logistic Regression model and the decision tree classifier

On the output for cell 5, the machine learning algorithm compared and contrasted the two main models that I had run in different cells. Both the **decision tree classification model** and the **Logistic regression model** were tried and tested, and after that i compared the **CV average mean** and decided to go with the model with the significantly higher mean accuracy which was the **decision tree model**.

The screenshot taken from the IDE below helps confirm the output which was received:

```
=====
DECISION TREE MODEL - COMPARISON & EVALUATION
=====
🔑 KEY DIFFERENCE: Decision Trees work on UNSCALED data (no scaling needed)

✓ Decision Tree trained on UNSCALED data

--- CROSS-VALIDATION (5-Fold Stratified) ---
Cross-Val Accuracy: 0.8455 ± 0.0116
Cross-Val F1-Score: 0.7960 ± 0.0146

=====
MODEL COMPARISON: LOGISTIC REGRESSION vs DECISION TREE
=====

Metric Logistic Regression Decision Tree
Accuracy                81.85%          85.85%
Balanced Accuracy        79.29%          85.50%
F1-Score                 0.7364          0.8117
CV Accuracy Mean         0.8154          0.8455

✓ BEST MODEL: Decision Tree (85.85% accuracy)
```

From the screenshot above, it is clear that the **Decision Tree Model** outperformed the **Logistic regression model**. And I believe the major reason for this happening is that the **relationship between the mechanical/electrical grid parameters and ultimate voltage stability hardly has any underlying linear correlations**.

Power systems are highly non linear in nature and the voltage stability will depend on many complex interactions between parameters like reactive power limits, rotor angles, load variance and so on.

What I mean is that, there is hardly any linear relationship between values and the states of the voltage stability within a system. So maybe after the tau values exceed a certain arithmetic



figure, we have a situation where the voltage level within the electrical grid system might become unstable, and whilst it's within a certain range, it remains stable.

So there seems to be a **nonlinear** relationship between the electrical grid values and the voltage stability across the system (grid)

PRACTICAL RECOMMENDATIONS GOING INTO THE FUTURE

So even though it's not common knowledge, it's good to know that most of the Voltage stability prediction models assume a 50% threshold for binary classification, but this brings about an issue.

In power systems, a **false negative (like missing a grid collapsing voltage spike)** is actually going to be catastrophic (financially and structurally in terms of damage). But however, a **false positive (unnecessary load/voltage shedding)** is merely an inconvenience which might even be a good thing (arguably). So based on the operational threshold analysis that was carried out in the code below:

```
1. Get the raw probabilities of being STABLE (Class 1)
probabilities = model.predict_proba(X_test_scaled)[: , 1]

# 2. Test multiple thresholds to find optimal trade-off
thresholds_to_test = [0.50, 0.60, 0.70, 0.75, 0.80, 0.90]

print("\nThreshold Analysis - Testing Different Safety Levels:\n")
print(f"{'Threshold':<12} {'Sensitivity':<12} {'Specificity':<12} {'Precision':<12} {'F1-Score':<12}")
print("-" * 60)

threshold_results = []

for threshold in thresholds_to_test:
    custom_pred = (probabilities >= threshold).astype(int)

    # Calculate metrics
    tn, fp, fn, tp = confusion_matrix(y_test, custom_pred).ravel()

    sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0 # True Positive Rate
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0 # True Negative Rate
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    f1_th = 2 * (precision * sensitivity) / (precision + sensitivity) if (precision +
sensitivity) > 0 else 0

    threshold_results.append({
```

Department of electrical/electronics



```
'Threshold': threshold,
'Sensitivity': sensitivity,
'Specificity': specificity,
'Precision': precision,
'F1': f1_th,
'Custom_Pred': custom_pred
})

print(f"{threshold:<12.2f} {sensitivity:<12.3f} {specificity:<12.3f} {precision:<12.3f}
{f1_th:<12.3f}")

# 3. Recommend optimal threshold
optimal_idx = np.argmax([r['F1'] for r in threshold_results])
optimal_threshold = threshold_results[optimal_idx]['Threshold']
print(f"\n✓ RECOMMENDED THRESHOLD: {optimal_threshold:.2f} (F1-Score:
{threshold_results[optimal_idx]['F1']:.4f})")

# ===== ISSUE #4: Use configurable NERC threshold from CONFIG =====
nerc_threshold = CONFIG['nerc_threshold']
custom_predictions = (probabilities >= nerc_threshold).astype(int)

print(f"\n" + "=" * 60)
print(f"NERC EARLY WARNING SYSTEM (Threshold = {nerc_threshold*100:.0f}%)")
print("=" * 60)
print("Interpretation: Grid must have ≥75% confidence to be declared SAFE")
print("Otherwise: EMERGENCY ACTION TRIGGERED\n")

nerc_cm = confusion_matrix(y_test, custom_predictions)
tn, fp, fn, tp = nerc_cm.ravel()

print("Classification Report:")
print(classification_report(y_test, custom_predictions,
                           target_names=['Emergency Action', 'Grid Safe']))

print(f"\nOperational Breakdown:")
print(f" True Negatives (Correct Emergency):  {tn:5d}  (avoiding false alarms)")
print(f" False Positives (False Alarms):      {fp:5d}  (unnecessary activations)")
print(f" False Negatives (MISSED RISKS):       {fn:5d}  ⚠ CRITICAL")
print(f" True Positives (Correct Safe):         {tp:5d}")

# 5. Visualize threshold impact
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Plot 1: Metrics vs Threshold
thresholds_plot = [r['Threshold'] for r in threshold_results]
sensitivity_plot = [r['Sensitivity'] for r in threshold_results]
specificity_plot = [r['Specificity'] for r in threshold_results]
f1_plot = [r['F1'] for r in threshold_results]
```



```
axes[0].plot(thresholds_plot, sensitivity_plot, marker='o', label='Sensitivity (TP Rate)',
linewidth=2)
axes[0].plot(thresholds_plot, specificity_plot, marker='s', label='Specificity (TN Rate)',
linewidth=2)
axes[0].plot(thresholds_plot, fl_plot, marker='^', label='F1-Score', linewidth=2)
axes[0].axvline(x=nerc_threshold, color='red', linestyle='--', linewidth=2, label=f'NERC
Threshold ({nerc_threshold})')
axes[0].set_xlabel('Probability Threshold')
axes[0].set_ylabel('Score')
axes[0].set_title('Threshold Optimization Analysis')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Plot 2: NERC Confusion Matrix
sns.heatmap(nerc_cm, annot=True, fmt='d', cmap='RdYlGn',
            xticklabels=['Emergency', 'Safe'],
            yticklabels=['Actually Unstable', 'Actually Stable'],
            ax=axes[1], cbar_kws={'label': 'Count'})
axes[1].set_title(f'NERC Threshold System (Threshold={nerc_threshold})')
axes[1].set_ylabel('True Label')
axes[1].set_xlabel('System Decision')

plt.tight_layout()
plt.show()
```

The model was able to compute the theoretical probability of being **Stable**, and then carrying out **thresh-hold testing** in order to find the optimal tradeoff.

After that was done, the metrics were calculated using **confusion matrices (all of these are in the code uploaded above)**.

After the model would have carried out **thresh-hold testing** and **confusion matrix calculations** then it will generally come up with a recommended optimal threshold after you run this code:

```
3. Recommend optimal threshold
optimal_idx = np.argmax([r['F1'] for r in threshold_results])
optimal_threshold = threshold_results[optimal_idx]['Threshold']
print(f"\n✓ RECOMMENDED THRESHOLD: {optimal_threshold:.2f} (F1-Score:
{threshold_results[optimal_idx]['F1']:.4f})")
```

Now this is where we use a configurable NERC threshold from Config:

```
nerc_threshold = CONFIG['nerc_threshold']
custom_predictions = (probabilities >= nerc_threshold).astype(int)

print(f"\n" + "=" * 60)
print(f"NERC EARLY WARNING SYSTEM (Threshold = {nerc_threshold*100:.0f}%)")
print("=" * 60)
print("Interpretation: Grid must have ≥75% confidence to be declared SAFE")
```

Department of electrical/electronics

Engineering



```
print("Otherwise: EMERGENCY ACTION TRIGGERED\n")

nec_cm = confusion_matrix(y_test, custom_predictions)
tn, fp, fn, tp = nec_cm.ravel()

print("Classification Report:")
print(classification_report(y_test, custom_predictions,
                           target_names=['Emergency Action', 'Grid Safe']))

print(f"\nOperational Breakdown:")
print(f" True Negatives (Correct Emergency): {tn:5d} (avoiding false alarms)")
print(f" False Positives (False Alarms): {fp:5d} (unnecessary activations)")
print(f" False Negatives (MISSED RISKS): {fn:5d} ⚠ CRITICAL")
print(f" True Positives (Correct Safe): {tp:5d}")
```

(All this code that i am uploading already happens to be in the larger upload that i made at the start of this paragraph).

Anyways, all this aside, the main recommendation that im making is that the NERC (Nigerian Electricity Regulatory Commission **set its new threshold to 0.75 or 75%**)

This means that the AI needs to be **at least 75% confident** that the grid voltages are stable before commencing normal operations.

If the model's stability prediction confidence level falls below 75%, there should be some sort of warning or **an Emergency Action** be triggered.

CONCLUSION

In conclusion, this particular project was able to successfully develop an AI-driven predictive model for power system voltage stability. The logistic regression model during the course of conducting this project and building this model proved to be a very robust tool, which was able to achieve a **81.85% accuracy but was clearly outperformed by the decision tree classifier that scored an accuracy of 85.85%.**

The analysis that were conducted in various cells of this IDE workspace confirmed that reaction times (τ) within the grid happen to be the major causes or variable that are responsible for instability within the electrical grid (**as i have already pointed out earlier in the report**) and in order to increase the confidence level , **i recommended that the operational decision**

Department of electrical/electronics

Engineering



threshold should be lifted to 75% so that this model could be optimised for actual real world application.

Limitations to this model:

The major limitation that this model has is that **it is designed primarily on a simulated 4-node grid that contains 1 generator and 3 primary receivers or consumers.** And this model is severely ill-equipped for the actual topological configuration of Nigeria's electrical grid system.

So the obvious recommendation for anyone audacious enough to even think about it would be to design a voltage stability prediction model for a grid system designed for millions and millions of nodes all across the federal Republic of Nigeria. data that had been scaled using part of the code shown above. This is the phase where the algorithm proceeds to use the 2000 samples of labelled data and attempts to then correctly label the remaining 8000 samples of unsampled data with a high enough accuracy level.

REFERENCES USED WHILST WRITING THIS REPORT

1. https://en.wikipedia.org/wiki/Logistic_regression *Logistic regression*. (n.d.). Wikipedia. Retrieved May 27, 2026.
2. https://en.wikipedia.org/wiki/Supervised_learning *supervised learning* (in the definition of Logistical regression). Wikipedia. Retrieved May 27, 2026.
3. https://en.wikipedia.org/wiki/Binary_classification *binary classification* (also in the definition of logistic regression). Wikipedia. Retrieved May 27, 2026.
4. https://en.wikipedia.org/wiki/Supervised_learning *Supervised learning* Wikipedia. Retrieved May 27, 2026.
5. <https://archive.ics.uci.edu/dataset/471/electrical+grid+stability+simulated+data> *UCI Electrical Grid Simulated multivariate Data* (the main dataset used to prime/build the machine learning model). Retrieved May 27, 2026.
6. <https://en.wikipedia.org/wiki/Statistics> *statistics* Wikipedia. Retrieved May 27, 2026.



7. https://en.wikipedia.org/wiki/Data_mining *data mining*. Wikipedia. Retrieved May 27, 2026.
8. https://en.wikipedia.org/wiki/Machine_learning *machine learning*. Wikipedia. Retrieved May 27, 2026.
9. https://en.wikipedia.org/wiki/Decision_tree *decision trees*. Wikipedia. Retrieved May 27, 2026.
10. https://en.wikipedia.org/wiki/Predictive_modelling *Predictive modelling*. Wikipedia. Retrieved May 27, 2026.
11. https://en.wikipedia.org/wiki/Statistical_classification *statistical classification*. Wikipedia. Retrieved May 27, 2026.
12. [https://en.wikipedia.org/wiki/Tree_\(abstract_data_type\)#Terminology](https://en.wikipedia.org/wiki/Tree_(abstract_data_type)#Terminology) *trees*. Wikipedia. Retrieved May 27, 2026.
13. https://en.wikipedia.org/wiki/Logical_conjunction *logical conjunction*. Wikipedia. Retrieved May 27, 2026.
14. https://en.wikipedia.org/wiki/Real_number *real numbers*. Wikipedia. Retrieved May 27, 2026.
15. https://en.wikipedia.org/wiki/Regression_analysis *regression analysis*. Wikipedia. Retrieved May 27, 2026

APPENDIX SECTION

1. **APPENDIX A:** The full annotated source code:

Cell 1:

```
from matplotlib import pyplot as plt
import pandas as pd
import seaborn as sns
import os
from sklearn.model_selection import train_test_split

# Ensure output directories exist
os.makedirs(CONFIG['model_dir'], exist_ok=True)
os.makedirs(CONFIG['output_dir'], exist_ok=True)
```



```

print("=" * 60)
print("DOWNLOADING DATASET")
print("=" * 60)
try:
    df = pd.read_csv(CONFIG['data_url'])
    print(f"✓ Dataset downloaded successfully")
except Exception as e:
    print(f"✗ ERROR downloading dataset: {e}")
    raise

print("\nValidating dataset...")
if df.shape[0] == 0:
    raise ValueError("Downloaded dataset is empty!")
if 'stabf' not in df.columns:
    raise ValueError("Required column 'stabf' not found in
dataset!")
print(f"✓ Dataset validation passed")

print("\n" + "=" * 60)
print("DATA PREPROCESSING & CLASS BALANCE CHECK")
print("=" * 60)

# Drop continuous 'stab', focus on 'stabf'
df = df.drop(columns=['stab'])
df['stabf'] = df['stabf'].map({'stable': 1, 'unstable': 0})

print("\n✓ Class Distribution BEFORE splitting:")
class_counts = df['stabf'].value_counts()
class_pct = df['stabf'].value_counts(normalize=True) * 100
for label, count in class_counts.items():
    pct = class_pct[label]
    status = "Stable" if label == 1 else "Unstable"
    print(f" {status:10} (Class {label}): {count:6} samples
({pct:5.2f}%)")

X = df.drop(columns=['stabf'])

```



```
y = df['stabf']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=CONFIG['test_size'],
    random_state=CONFIG['random_state'], stratify=y
)

print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples")

print("\n" + "=" * 60)
print("FEATURE CORRELATION WITH STABILITY")
print("=" * 60)
correlation_with_stability =
X.corrwith(y).sort_values(ascending=False)
print("\nCorrelation with Stability (1=Stable, 0=Unstable):")
print(correlation_with_stability)

plt.figure(figsize=(12, 8))
correlation_matrix = pd.concat([X, y], axis=1).corr()
sns.heatmap(correlation_matrix, annot=True, fmt='.2f',
    cmap='coolwarm', center=0, cbar_kws={'label': 'Correlation'})
plt.title('Feature Correlation Matrix with Stability Target')
plt.tight_layout()
plt.show()
```

Cell 2:

```
print("\n" + "=" * 60)
print("DATA SCALING (for distance-based algorithms)")
print("=" * 60)
print("StandardScaler centers features to mean=0, std=1")
print("This is REQUIRED for Logistic Regression but NOT for
Decision Trees\n")

scaler = StandardScaler()
```



```
X_train_scaled_arr = scaler.fit_transform(X_train)
X_test_scaled_arr = scaler.transform(X_test)

# ===== ISSUE #3: Preserve feature names after scaling
=====
X_train_scaled = pd.DataFrame(X_train_scaled_arr,
                              columns=X_train.columns, index=X_train.index)
X_test_scaled = pd.DataFrame(X_test_scaled_arr,
                              columns=X_test.columns, index=X_test.index)

print("✓ Scaling complete with feature names preserved")
print(f" Scaled training data shape: {X_train_scaled.shape}")
print(f" Scaled testing data shape: {X_test_scaled.shape}")
print(f" Features: {list(X_train_scaled.columns)}")
print(f" Sample scaled values (first row):
{X_train_scaled.iloc[0, :3].values}")
```

Cell 3:

```
import seaborn as sns

print("\n" + "=" * 60)
print("LOGISTIC REGRESSION - TRAINING & EVALUATION")
print("=" * 60)

# 1. Initialize and Train the Model
model = LogisticRegression(random_state=CONFIG['random_state'],
                             max_iter=1000)
model.fit(X_train_scaled, y_train)

print("✓ Model trained successfully")

# 2. CROSS-VALIDATION: Test generalization (K-Fold)
print(f"\n--- CROSS-VALIDATION ({CONFIG['cv_splits']}-Fold
Stratified) ---")
```

```

cv_strategy = StratifiedKFold(n_splits=CONFIG['cv_splits'],
                               shuffle=True, random_state=CONFIG['random_state'])
cv_scores = cross_val_score(model, X_train_scaled, y_train,
                              cv=cv_strategy,
                              scoring='accuracy')
cv_f1 = cross_val_score(model, X_train_scaled, y_train,
                          cv=cv_strategy,
                          scoring='f1')
print(f"Cross-Val Accuracy: {cv_scores.mean():.4f} ±
{cv_scores.std():.4f}")
print(f"Cross-Val F1-Score: {cv_f1.mean():.4f} ±
{cv_f1.std():.4f}")
print(f"Individual fold accuracies: {[f'{s:.4f}' for s in
cv_scores]}")

# 3. Make Predictions on the unseen 20% of data
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:, 1] # Probability
of being stable

# 4. Calculate Multiple Metrics
accuracy = accuracy_score(y_test, y_pred)
balanced_acc = balanced_accuracy_score(y_test, y_pred) # Better
for imbalanced data
f1 = f1_score(y_test, y_pred)

print("\n" + "=" * 60)
print("TEST SET PERFORMANCE")
print("=" * 60)
print(f"Accuracy: {accuracy * 100:6.2f}%")
print(f"Balanced Accuracy: {balanced_acc * 100:6.2f}% (better
for imbalanced)")
print(f"F1-Score: {f1:.4f} (harmonic mean of precision
& recall)")

print("\nDetailed Classification Report:")

```



```
print(classification_report(y_test, y_pred,
                             target_names=['Unstable (0)', 'Stable (1)']))

# 5. CONFUSION MATRIX - Visual breakdown
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(f"                Predicted")
print(f"                Unstable  Stable")
print(f"Actual Unstable   {cm[0,0]:5d}   {cm[0,1]:5d}")
print(f"Actual Stable    {cm[1,0]:5d}   {cm[1,1]:5d}")

# Confusion Matrix Visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Confusion Matrix Heatmap
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=['Unstable', 'Stable'],
             yticklabels=['Unstable', 'Stable'],
             cbar_kws={'label': 'Count'}, ax=axes[0])
axes[0].set_title('Confusion Matrix - Logistic Regression')
axes[0].set_ylabel('True Label')
axes[0].set_xlabel('Predicted Label')

# Plot 2: ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

axes[1].plot(fpr, tpr, color='darkorange', lw=2.5, label=f'ROC
curve (AUC = {roc_auc:.3f})')
axes[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--',
             label='Random Classifier')
axes[1].fill_between(fpr, tpr, alpha=0.2, color='darkorange')
axes[1].set_xlim([0.0, 1.0])
axes[1].set_ylim([0.0, 1.05])
axes[1].set_xlabel('False Positive Rate (False Alarms)')
axes[1].set_ylabel('True Positive Rate (Correct Detections)')
axes[1].set_title('ROC Curve - Grid Stability Prediction')
```

```
axes[1].legend(loc="lower right")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 6. PRECISION-RECALL CURVE (Alternative to ROC)
precision, recall, _ = precision_recall_curve(y_test, y_prob)
plt.figure(figsize=(10, 6))
plt.plot(recall, precision, marker='o', label=f'PR curve
(F1={f1:.3f})')
plt.xlabel('Recall (True Positive Rate)')
plt.ylabel('Precision (True Positive / Predicted Positive)')
plt.title('Precision-Recall Curve - Grid Stability')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Cell 4

```
import numpy as np

print("\n" + "=" * 60)
print("FEATURE IMPORTANCE & GRID VULNERABILITY ANALYSIS")
print("=" * 60)

# 1. Extract the coefficients (weights) from your trained model
coefficients = model.coef_[0]
feature_names = X.columns
intercept = model.intercept_[0]

# 2. Create a detailed table
feature_importance = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': coefficients,
    'Abs_Coefficient': np.abs(coefficients)
})

# 3. Sort by absolute impact
```

```

feature_importance =
feature_importance.sort_values(by='Abs_Coefficient',
ascending=False)

print("\nFeature Importance Ranking:")
print("(Positive = Stabilizing | Negative = Destabilizing)\n")
for idx, row in feature_importance.iterrows():
    impact_type = "STABILIZING " if row['Coefficient'] > 0 else
"DESTABILIZING"
    print(f"{row['Feature']:10} | Coef: {row['Coefficient']:8.4f}
| {impact_type} | Impact: {row['Abs_Coefficient']:8.4f}")

print(f"\nModel Intercept (bias): {intercept:.4f}")

# ===== ISSUE #8: Export feature importance to CSV =====
importance_path = os.path.join(CONFIG['output_dir'],
'feature_importance_logistic.csv')
feature_importance.to_csv(importance_path, index=False)
print(f"\n✓ Feature importance saved to: {importance_path}")

# 4. Visualization
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: Sorted bar plot
colors = ['green' if c > 0 else 'red' for c in
feature_importance['Coefficient']]
axes[0].barh(feature_importance['Feature'],
feature_importance['Coefficient'], color=colors, alpha=0.7)
axes[0].set_xlabel('Coefficient Value')
axes[0].set_title('Feature Importance (Logistic
Regression)\nGreen=Stabilizing, Red=Destabilizing')
axes[0].axvline(x=0, color='black', linestyle='--', linewidth=0.8)
axes[0].grid(True, alpha=0.3, axis='x')

# Plot 2: Absolute importance
axes[1].barh(feature_importance['Feature'],
feature_importance['Abs_Coefficient'],
color='steelblue', alpha=0.7)
axes[1].set_xlabel('Absolute Impact')

```



```
axes[1].set_title('Feature Importance by Absolute Impact\n(Higher  
= More Critical to Decision)')  
axes[1].grid(True, alpha=0.3, axis='x')  
  
plt.tight_layout()  
plt.show()  
  
# 5. Top 3 Grid Parameters to Monitor  
print("\n" + "=" * 60)  
print("💡 TOP 3 CRITICAL PARAMETERS FOR GRID STABILITY")  
print("=" * 60)  
top_3 = feature_importance.head(3)  
for rank, (idx, row) in enumerate(top_3.iterrows(), 1):  
    print(f"{rank}. {row['Feature'][:15]} (Impact:  
{row['Abs_Coefficient']:.4f})")
```

Cell 5

```
import pandas as pd  
import seaborn as sns  
import matplotlib.pyplot as plt  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.model_selection import cross_val_score  
from sklearn.metrics import accuracy_score,  
balanced_accuracy_score, f1_score, classification_report,  
confusion_matrix, roc_curve, auc  
import os  
  
print("\n" + "=" * 60)  
print("DECISION TREE MODEL - COMPARISON & EVALUATION")  
print("=" * 60)  
print("🔑 KEY DIFFERENCE: Decision Trees work on UNSCALED data (no  
scaling needed)\n")  
  
# 1. Initialize the Decision Tree with CONFIG parameters  
dt_model = DecisionTreeClassifier(  
    max_depth=CONFIG['dt_max_depth'],  
    random_state=CONFIG['random_state'],  
    min_samples_split=CONFIG['dt_min_samples_split'],  
    min_samples_leaf=CONFIG['dt_min_samples_leaf'],
```

Department of electrical/electronics


```

class_weight='balanced'
)

# 2. Train on UNSCALED data
dt_model.fit(X_train, y_train)
print("✓ Decision Tree trained on UNSCALED data")

# 3. CROSS-VALIDATION
print(f"\n--- CROSS-VALIDATION ({CONFIG['cv_splits']}-Fold
Stratified) ---")
dt_cv_scores = cross_val_score(dt_model, X_train, y_train,
cv=cv_strategy, scoring='accuracy')
dt_cv_f1 = cross_val_score(dt_model, X_train, y_train,
cv=cv_strategy, scoring='f1')
print(f"Cross-Val Accuracy: {dt_cv_scores.mean():.4f} ±
{dt_cv_scores.std():.4f}")
print(f"Cross-Val F1-Score: {dt_cv_f1.mean():.4f} ±
{dt_cv_f1.std():.4f}")

# 4. Predictions
dt_pred = dt_model.predict(X_test)
dt_prob = dt_model.predict_proba(X_test)[: , 1]

# 5. Metrics
dt_accuracy = accuracy_score(y_test, dt_pred)
dt_balanced_acc = balanced_accuracy_score(y_test, dt_pred)
dt_f1 = f1_score(y_test, dt_pred)

# 6. Comparison Table
print("\n" + "=" * 60)
print("MODEL COMPARISON: LOGISTIC REGRESSION vs DECISION TREE")
print("=" * 60)

comparison = pd.DataFrame({
    'Metric': ['Accuracy', 'Balanced Accuracy', 'F1-Score', 'CV
Accuracy Mean'],
    'Logistic Regression': [
        f"{accuracy * 100:.2f}%",
        f"{balanced_acc * 100:.2f}%",

```

```

        f"{f1:.4f}",
        f"{cv_scores.mean():.4f}"
    ],
    'Decision Tree': [
        f"{dt_accuracy * 100:.2f}%",
        f"{dt_balanced_acc * 100:.2f}%",
        f"{dt_f1:.4f}",
        f"{dt_cv_scores.mean():.4f}"
    ]
])
print("\n" + comparison.to_string(index=False))

# Best model determination
if accuracy > dt_accuracy:
    best_model = "Logistic Regression"
    best_accuracy = accuracy
else:
    best_model = "Decision Tree"
    best_accuracy = dt_accuracy

print(f"\n✓ BEST MODEL: {best_model} ({best_accuracy * 100:.2f}% accuracy)")

# Plots
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

cm_lr = confusion_matrix(y_test, y_pred)
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Blues', ax=axes[0, 0])
axes[0, 0].set_title(f'Logistic Regression (Acc: {accuracy*100:.2f}%)')

cm_dt = confusion_matrix(y_test, dt_pred)
sns.heatmap(cm_dt, annot=True, fmt='d', cmap='Oranges', ax=axes[0, 1])
axes[0, 1].set_title(f'Decision Tree (Acc: {dt_accuracy*100:.2f}%)')

fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob)

```

```
axes[1, 0].plot(fpr_lr, tpr_lr, label=f'LR (AUC = {auc(fpr_lr,
tpr_lr):.3f})')
axes[1, 0].set_title('ROC - Logistic Regression')

fpr_dt, tpr_dt, _ = roc_curve(y_test, dt_prob)
axes[1, 1].plot(fpr_dt, tpr_dt, color='green', label=f'DT (AUC =
{auc(fpr_dt, tpr_dt):.3f})')
axes[1, 1].set_title('ROC - Decision Tree')

plt.tight_layout()
plt.show()

# Feature Importance
dt_feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("\n--- DECISION TREE FEATURE IMPORTANCE ---")
print(dt_feature_importance.head(5))

dt_importance_path = os.path.join(CONFIG['output_dir'],
'feature_importance_decision_tree.csv')
dt_feature_importance.to_csv(dt_importance_path, index=False)
```

Cell 6:

```
import numpy as np
import joblib
from sklearn.metrics import confusion_matrix,
classification_report, roc_curve, auc, precision_recall_curve
import matplotlib.pyplot as plt
import seaborn as sns
import os

print("\n" + "=" * 60)
print("OPERATIONAL THRESHOLD ANALYSIS - NERC EARLY WARNING
SYSTEM")
print("=" * 60)
```

```
# 1. Get the raw probabilities of being STABLE (Class 1)
probabilities = model.predict_proba(X_test_scaled)[: , 1]

# 2. Test multiple thresholds to find optimal trade-off
thresholds_to_test = [0.50, 0.60, 0.70, 0.75, 0.80, 0.90]

print("\nThreshold Analysis - Testing Different Safety Levels:\n")
print(f"{'Threshold':<12} {'Sensitivity':<12} {'Specificity':<12} {'Precision':<12} {'F1-Score':<12}")
print("-" * 60)

threshold_results = []

for threshold in thresholds_to_test:
    custom_pred = (probabilities >= threshold).astype(int)

    # Calculate metrics
    tn, fp, fn, tp = confusion_matrix(y_test, custom_pred).ravel()

    sensitivity = tp / (tp + fn) if (tp + fn) > 0 else 0 # True Positive Rate
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0 # True Negative Rate
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    f1_th = 2 * (precision * sensitivity) / (precision + sensitivity) if (precision + sensitivity) > 0 else 0

    threshold_results.append({
        'Threshold': threshold,
        'Sensitivity': sensitivity,
        'Specificity': specificity,
        'Precision': precision,
        'F1': f1_th,
        'Custom_Pred': custom_pred
    })

    print(f"{'threshold':<12.2f} {'sensitivity':<12.3f} {'specificity':<12.3f} {'precision':<12.3f} {'f1_th':<12.3f}")
```

```
# 3. Recommend optimal threshold
optimal_idx = np.argmax([r['F1'] for r in threshold_results])
optimal_threshold = threshold_results[optimal_idx]['Threshold']
print(f"\n✓ RECOMMENDED THRESHOLD: {optimal_threshold:.2f}
(F1-Score: {threshold_results[optimal_idx]['F1']:.4f})")

# ===== ISSUE #4: Use configurable NERC threshold from CONFIG
=====
nerc_threshold = CONFIG['nerc_threshold']
custom_predictions = (probabilities >= nerc_threshold).astype(int)

print(f"\n" + "=" * 60)
print(f"NERC EARLY WARNING SYSTEM (Threshold =
{nerc_threshold*100:.0f}%)")
print("=" * 60)
print("Interpretation: Grid must have ≥75% confidence to be
declared SAFE")
print("Otherwise: EMERGENCY ACTION TRIGGERED\n")

nerc_cm = confusion_matrix(y_test, custom_predictions)
tn, fp, fn, tp = nerc_cm.ravel()

print("Classification Report:")
print(classification_report(y_test, custom_predictions,
                           target_names=['Emergency Action', 'Grid
Safe']))

print(f"\nOperational Breakdown:")
print(f" True Negatives (Correct Emergency): {tn:5d} (avoiding
false alarms)")
print(f" False Positives (False Alarms): {fp:5d}
(unnecessary activations)")
print(f" False Negatives (MISSED RISKS): {fn:5d} ⚠
CRITICAL")
print(f" True Positives (Correct Safe): {tp:5d}")

# 5. Visualize threshold impact
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
```

```
# Plot 1: Metrics vs Threshold
thresholds_plot = [r['Threshold'] for r in threshold_results]
sensitivity_plot = [r['Sensitivity'] for r in threshold_results]
specificity_plot = [r['Specificity'] for r in threshold_results]
f1_plot = [r['F1'] for r in threshold_results]

axes[0].plot(thresholds_plot, sensitivity_plot, marker='o',
label='Sensitivity (TP Rate)', linewidth=2)
axes[0].plot(thresholds_plot, specificity_plot, marker='s',
label='Specificity (TN Rate)', linewidth=2)
axes[0].plot(thresholds_plot, f1_plot, marker='^',
label='F1-Score', linewidth=2)
axes[0].axvline(x=nerc_threshold, color='red', linestyle='--',
linewidth=2, label=f'NERC Threshold ({nerc_threshold})')
axes[0].set_xlabel('Probability Threshold')
axes[0].set_ylabel('Score')
axes[0].set_title('Threshold Optimization Analysis')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Plot 2: NERC Confusion Matrix
sns.heatmap(nerc_cm, annot=True, fmt='d', cmap='RdYlGn',
xticklabels=['Emergency', 'Safe'],
yticklabels=['Actually Unstable', 'Actually Stable'],
ax=axes[1], cbar_kws={'label': 'Count'})
axes[1].set_title(f'NERC Threshold System
(Threshold={nerc_threshold})')
axes[1].set_ylabel('True Label')
axes[1].set_xlabel('System Decision')

plt.tight_layout()
plt.show()

# ===== ISSUE #10: Add prediction metadata with confidence
intervals =====
print("\n" + "=" * 60)
print("PREDICTION METADATA & CONFIDENCE ANALYSIS")
print("=" * 60)
```

```
# Create prediction dataframe with confidence info
predictions_metadata = pd.DataFrame({
    'Actual_Stability': y_test.values,
    'Predicted_Stability': y_pred,
    'Prediction_Confidence': np.maximum(y_prob, 1 - y_prob), #
    Confidence of the prediction
    'Probability Stable': y_prob,
    'Correct_Prediction': (y_test.values == y_pred).astype(int)
})

print("\nSample Predictions with Confidence Metrics:")
print(predictions_metadata.head(10).to_string())

# Analyze confidence distribution
low_conf = (predictions_metadata['Prediction_Confidence'] <
0.6).sum()
med_conf = ((predictions_metadata['Prediction_Confidence'] >= 0.6)
&
              (predictions_metadata['Prediction_Confidence'] <
0.8)).sum()
high_conf = (predictions_metadata['Prediction_Confidence'] >=
0.8).sum()

print(f"\nConfidence Distribution:")
print(f"  Low Confidence (<60%): {low_conf:5d} predictions")
print(f"  Medium Confidence:      {med_conf:5d} predictions")
print(f"  High Confidence (≥80%): {high_conf:5d} predictions")

# Save metadata
metadata_path = os.path.join(CONFIG['output_dir'],
'prediction_metadata.csv')
predictions_metadata.to_csv(metadata_path, index=False)
print(f"\n✓ Prediction metadata saved to: {metadata_path}")

# ===== ISSUE #2 & #5: Use CONFIG for model paths and
validate before saving =====
print("\n" + "=" * 60)
print("MODEL PERSISTENCE - SAVING FOR PRODUCTION")
print("=" * 60)
```

```

model_path = os.path.join(CONFIG['model_dir'],
                             'voltage_stability_model.pkl')
scaler_path = os.path.join(CONFIG['model_dir'],
                             'voltage_scaler.pkl')
config_path = os.path.join(CONFIG['model_dir'], 'config.pkl')

# Calculate ROC AUC for saving
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_prob)
roc_auc_lr = auc(fpr_lr, tpr_lr)

# ===== ISSUE #5: Validate models before saving =====
print("\nValidating models before persistence...")
try:
    # Test predictions
    test_pred = model.predict(X_test_scaled)
    test_prob = model.predict_proba(X_test_scaled)

    # Validate scaler
    test_scaled = scaler.transform(X_test)

    if test_pred.shape[0] > 0 and test_scaled.shape[0] > 0:
        print("✓ Model validation passed")
    else:
        raise ValueError("Model validation failed: empty
predictions")
except Exception as e:
    print(f"✗ Model validation failed: {e}")
    raise

# Save models
joblib.dump(model, model_path)
joblib.dump(scaler, scaler_path)
joblib.dump(CONFIG, config_path)

print(f"✓ Model saved to: {model_path}")
print(f"✓ Scaler saved to: {scaler_path}")
print(f"✓ Config saved to: {config_path}")

```




```
# Save all metrics summary
metrics_summary = {
    'Model': 'Logistic Regression',
    'Test_Accuracy': accuracy,
    'Test_F1_Score': f1,
    'Test_Balanced_Accuracy': balanced_acc,
    'CV_Accuracy_Mean': cv_scores.mean(),
    'CV_Accuracy_Std': cv_scores.std(),
    'ROC_AUC': roc_auc_lr,
    'Training_Samples': X_train.shape[0],
    'Test_Samples': X_test.shape[0]
}

metrics_path = os.path.join(CONFIG['output_dir'],
                             'model_metrics.pkl')
joblib.dump(metrics_summary, metrics_path)
print(f"✓ Metrics saved to: {metrics_path}")
```

2. **APPENDIX 2:** Raw Data sample (the first 20 lines of the code)

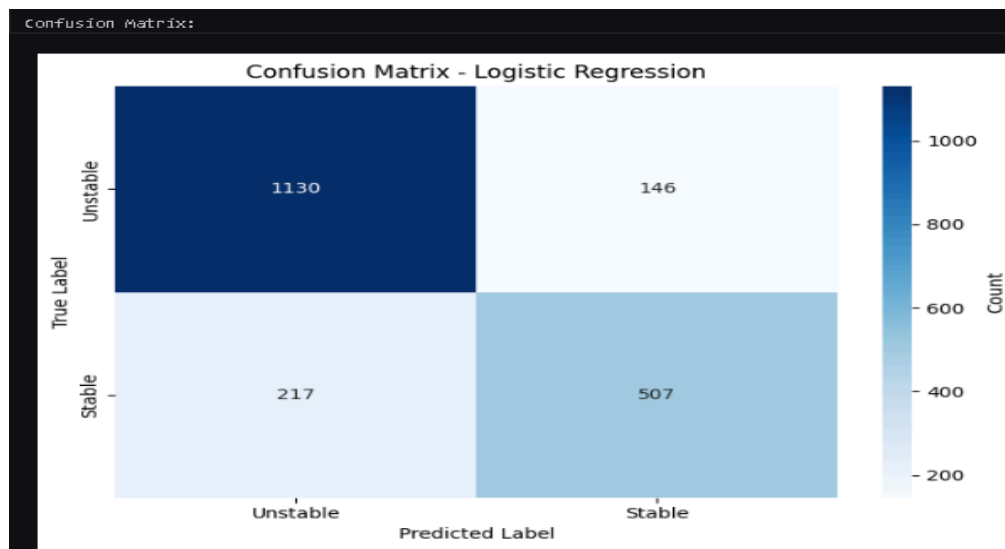
<https://archive.ics.uci.edu/dataset/471/electrical+grid+stability+simulated+data>

```
"tau1", "tau2", "tau3", "tau4", "p1", "p2", "p3", "p4", "g1", "g2", "g3", "g4", "stab", "stabf"
2.95906002455997, 3.07988520422811, 8.38102539191882, 9.78075443222607, 3.76308477206316,
-0.782603630987543, -1.25739482958732, -1.7230863114883, 0.650456460887227, 0.8595781057
52345, 0.887444920638513, 0.958033987602737, 0.0553474891727752, "unstable"
9.3040972346785, 4.90252411201167, 3.04754072762177, 1.36935735529605, 5.06781210427845, -
1.94005842705193, -1.87274168559721, -1.25501199162931, 0.41344056837935, 0.862414076352
903, 0.562139050527675, 0.781759910653126, -0.00595746432603695, "stable"
8.97170690932022, 8.84842842134833, 3.04647874898866, 1.21451813833956, 3.40515818001095,
-1.20745559234302, -1.27721014673295, -0.92049244093498, 0.163041039311334, 0.766886565
26962, 0.839444015400588, 0.109853244952427, 0.00347087904838871, "unstable"
0.716414776295121, 7.66959964406565, 4.48664083058949, 2.34056298396795, 3.96379106326633
, -1.02747330413905, -1.9389441526466, -0.997373606480681, 0.446208906537321, 0.976744082
924302, 0.929380522872661, 0.36271777426931, 0.028870543444887, "unstable"
3.13411155161342, 7.60877161603408, 4.94375930178099, 9.85757326996638, 3.52581081652096,
-1.12553095451115, -1.84597485447561, -0.554305007534195, 0.797109525792467, 0.455449947
148291, 0.656946658473716, 0.820923486481631, 0.0498603734837059, "unstable"
6.99920870915215, 9.10924671643764, 3.7840659093988, 4.26778760341917, 4.42966882280625, -
1.85713917685067, -0.670396827477973, -1.90213281847761, 0.261792854117035, 0.0779296721
686504, 0.542883858522323, 0.469931041283024, -0.0173849069339299, "stable"
6.71016636762265, 3.76520418352323, 6.92931443867021, 8.81856192345691, 2.39741922937942,
-0.614589561907959, -1.20882589139371, -0.574003776077751, 0.177889561089242, 0.39797658
0119618, 0.402045952122677, 0.376630279179848, 0.0059535705684696, "unstable"
```

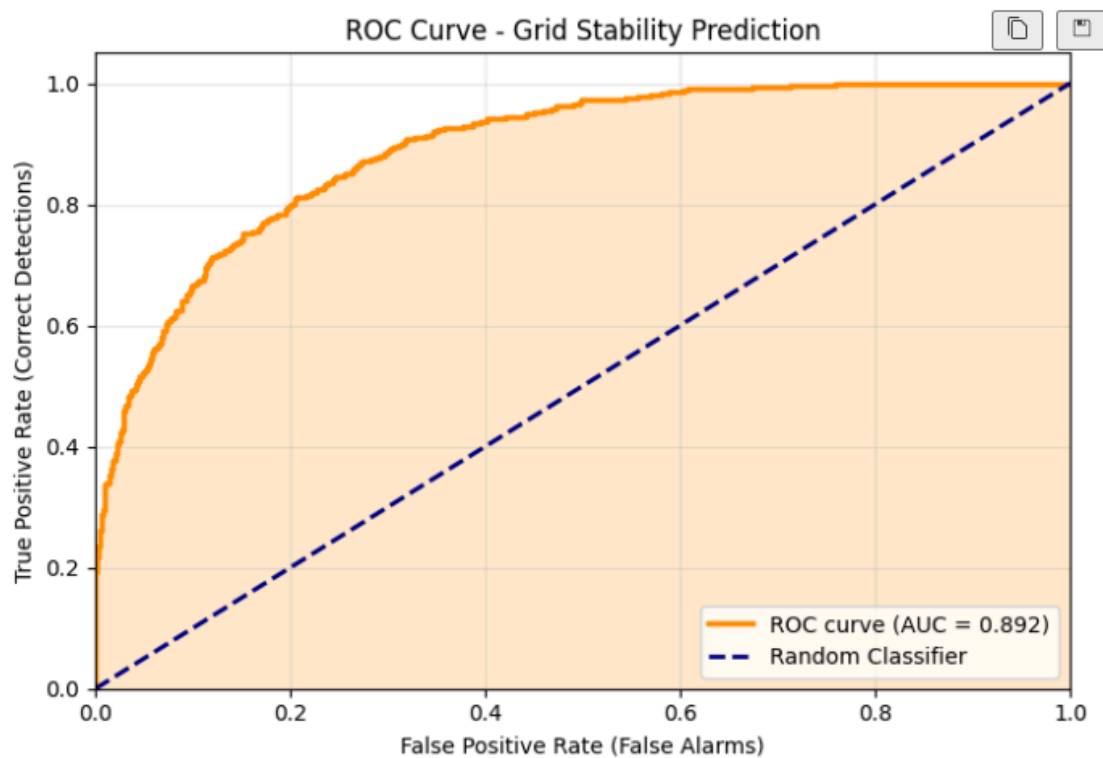


```
6.95351155782518,1.37912500228657,5.71940003020014,7.87030742603241,3.22449515790325,
-0.748997751247627,-1.1865169283662,-1.28898047828942,0.371384852592056,0.6332035554
33068,0.732740894287254,0.380544478575059,0.0166342974538163,"unstable"
4.68985192749584,4.00774746209312,1.47857287590383,3.73378749230253,4.04129961368168,
-1.41034431234687,-1.23820427574315,-1.39275102559166,0.269708270661235,0.2503642172
02032,0.164941118523114,0.482439411001001,-0.0386765836142913,"stable"
9.84149648830214,1.41382159178999,9.76985554870019,7.64161626330592,4.72759504201898,
-1.99136336778711,-0.857637206946418,-1.87859446728545,0.376355980566834,0.544415315
810004,0.792039424640656,0.116262822352385,0.0123834141734175,"unstable"
5.93011011760085,6.73087337437817,6.24513790053774,0.53328751282593,2.32709159288717,
-0.702501235021046,-1.11691962966126,-0.507670728204865,0.239815632938209,0.56310959
3653352,0.164461028193514,0.753701355880644,-0.0284114612679547,"stable"
5.38129925652915,8.01452101703768,8.0951741572488,6.76924769920496,5.50755100504513,-
1.97271385420545,-1.84933257978301,-1.68550457105667,0.359974025112588,0.17356881660
0156,0.349143850920014,0.628860423786381,0.0281301999792468,"unstable"
1.61678683711417,2.93922810703424,0.819791186154762,4.19180384910867,3.7522819142677,
-1.48488454526459,-1.28058145097331,-0.986815918029798,0.899698249387734,0.866545796
305834,0.303920577803025,0.0776101261113305,-0.0486167008032998,"stable"
8.55159820963844,8.31495222344878,2.54996382964303,9.92680728110744,4.89171373543719,
-1.80862626682759,-1.16706123460778,-1.91602623400182,0.612403529208615,0.2809828575
20184,0.354342487890125,0.472192077743488,0.0277556924227216,"unstable"
1.13210759360397,2.92032402330491,8.95107854465566,7.24858347796639,5.03368055334748,
-1.84607954217829,-1.36277740864708,-1.82482360252211,0.352291990740888,0.5241728822
49252,0.599003855980165,0.674389868319682,0.0148804322017331,"unstable"
7.02136199991397,4.37429363886033,4.77590415834652,8.83842594831276,3.33585746131655,
-0.962391781186988,-1.40762695274189,-0.96583872738767,0.711099508262505,0.625363823
866651,0.468334969702021,0.895143061005827,0.0725083555585227,"unstable"
4.95224090481821,8.08867235232354,8.88331865624976,5.6945572196255,5.06729577625163,-
1.68141419491175,-1.87706128759659,-1.50882029374329,0.305662026508872,0.30790384193
7671,0.889893892795722,0.879427907898293,0.065617204379854,"unstable"
4.1428304011016,2.43908920766944,1.29045594015738,9.45644318889404,3.93479591772548,-
1.46929869766647,-1.766941258873,-0.698555961186008,0.800756745514446,0.840807129209
329,0.917833435936101,0.793981811714133,-0.00696582814861133,"stable"
9.34612594367674,7.92002984757187,2.33527630767662,3.26918132842386,4.58117391713639,
-1.10675268019072,-1.74707763942746,-1.72734359751821,0.836076237689777,0.7132535780
90513,0.161517853668334,0.515983284965043,0.0413744064662021,"unstable"
```

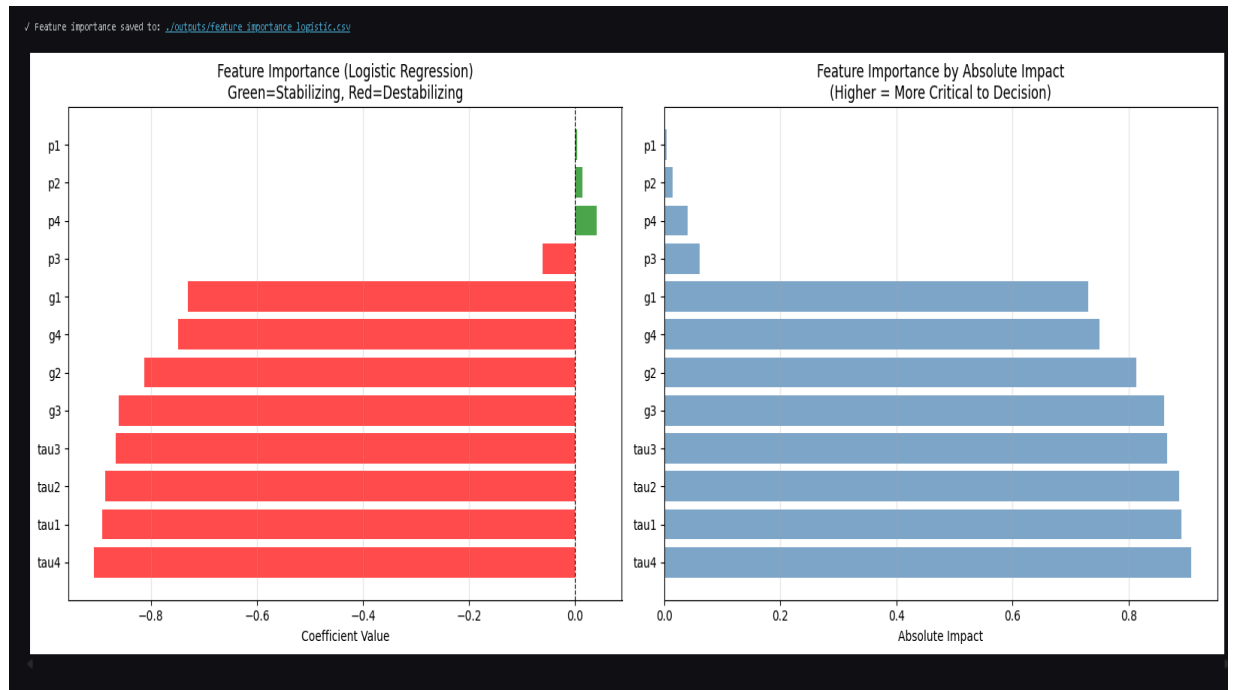
3. APPENDIX C: Confusion Matrix for the logistic regression model



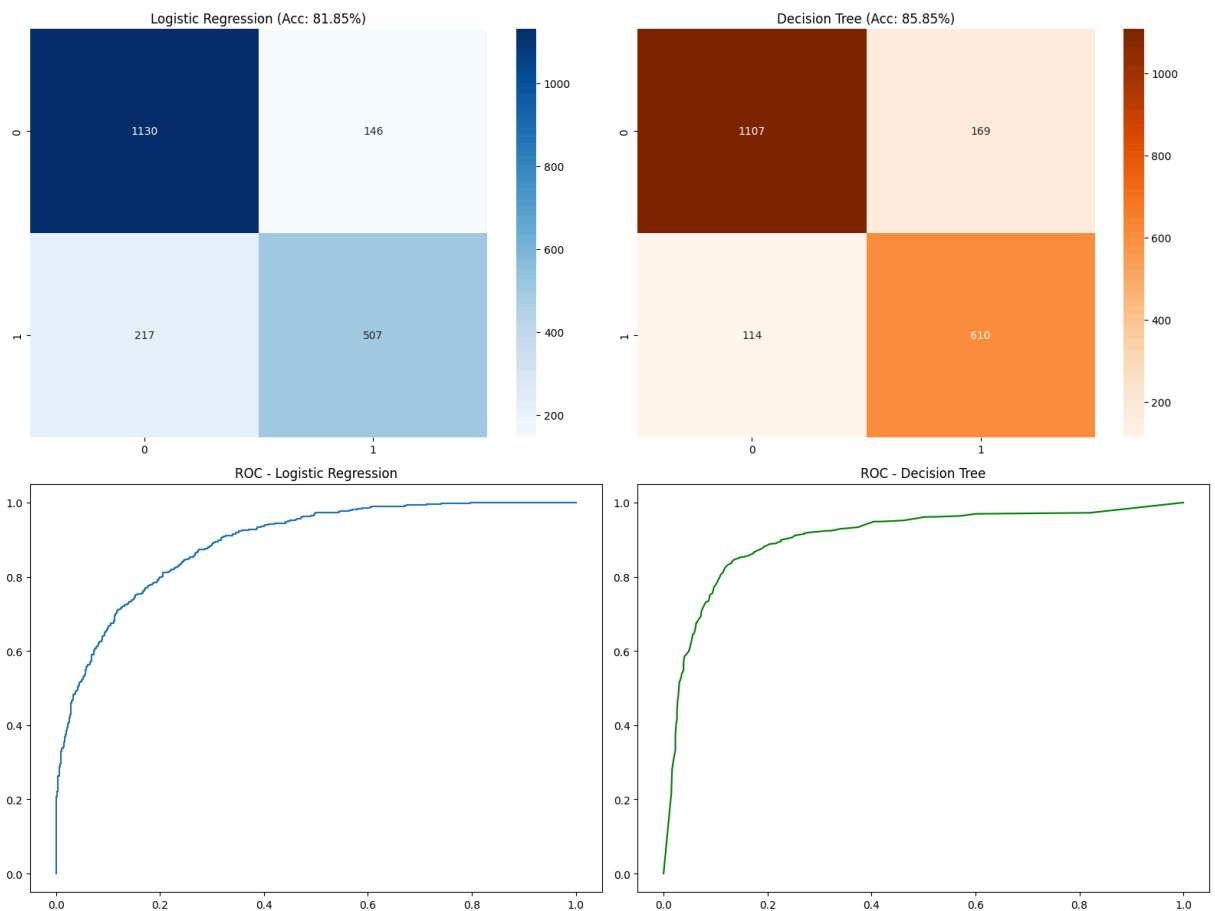
4. **APPENDIX D:** ROC and AUC curves for the voltage stability machine learning model.



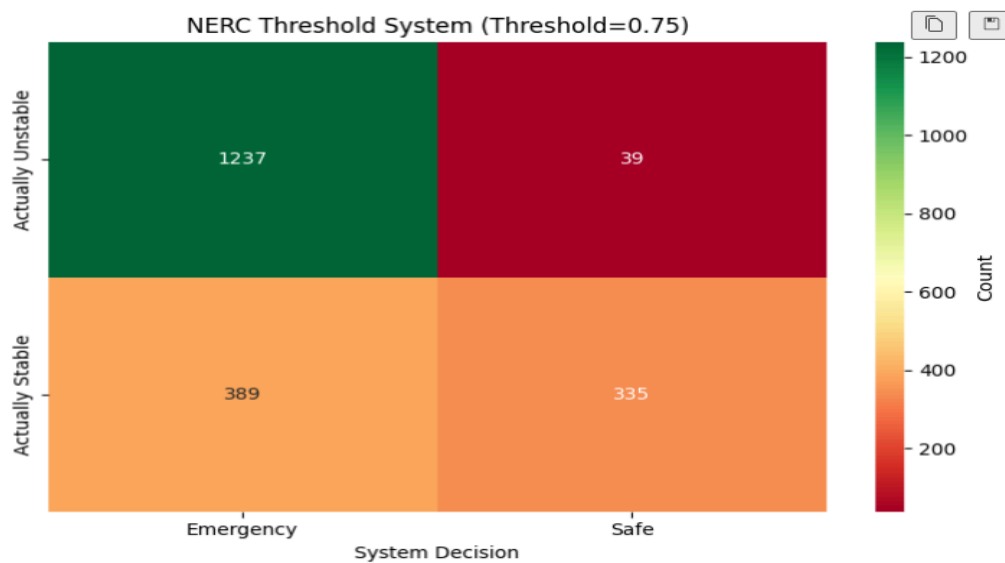
5. **APPENDIX E :** Feature importance for logistic regression, feature importance by absolute impact (higher means that they are more critical to decision)



6. **RECEIVER OPERATING CHARACTERISTICS CURVE:**



7. NERC THRESHOLD SYSTEM (Threshold = 0.75)

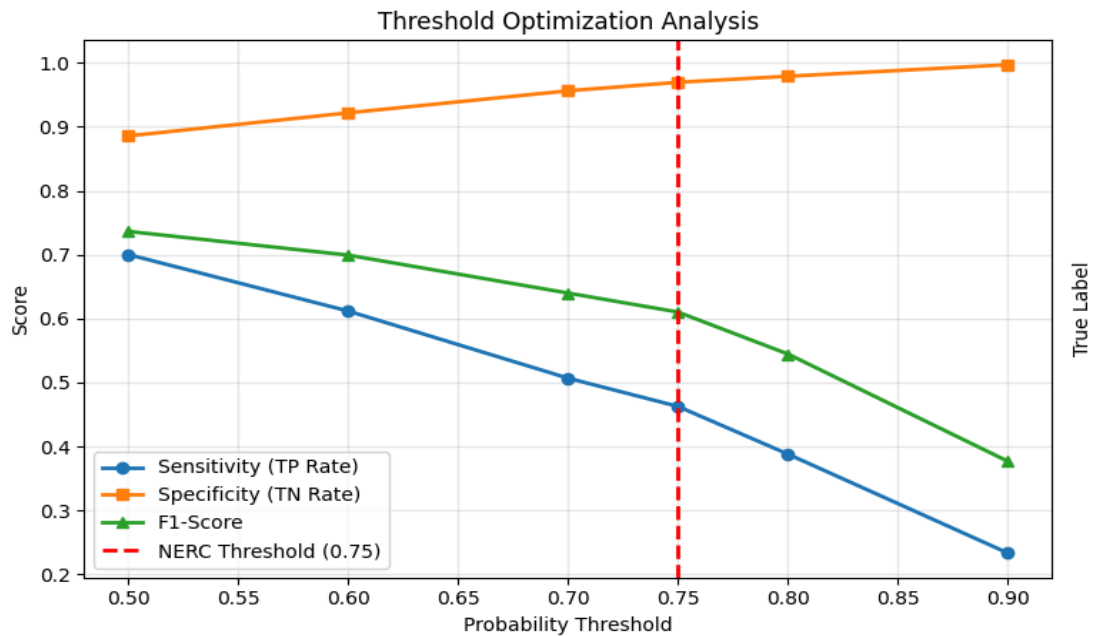


8. THRESHOLD OPTIMIZATION ANALYSIS:

Department of electrical/electronics

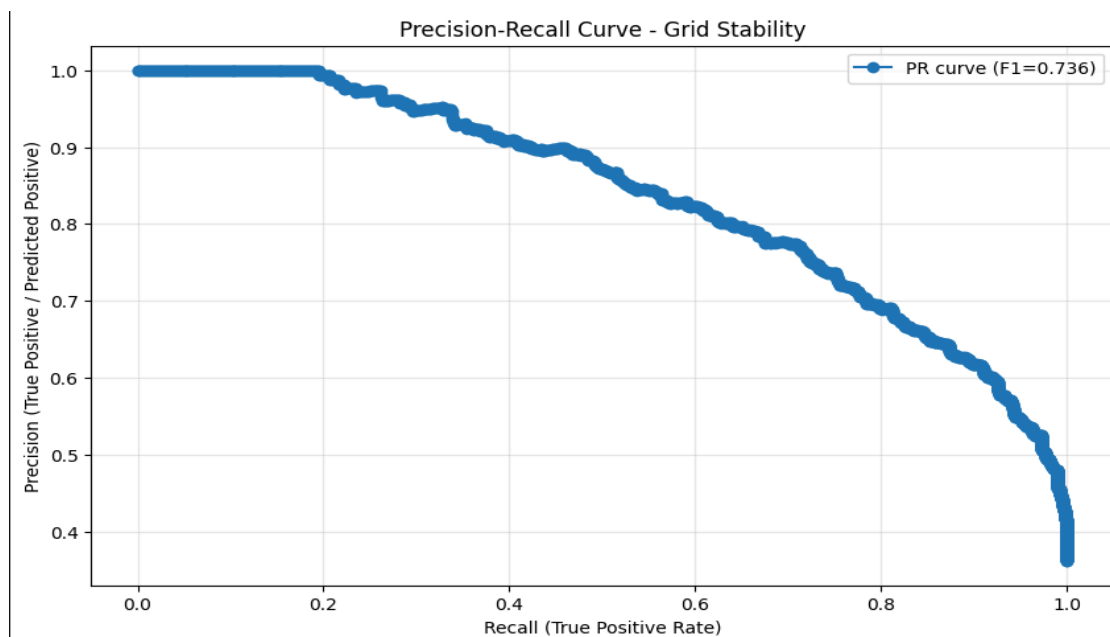
Engineering

This is the graph that contains sensitivity (TP Rate), Specificity (TN Rate), F1-Score, and NERC Threshold.



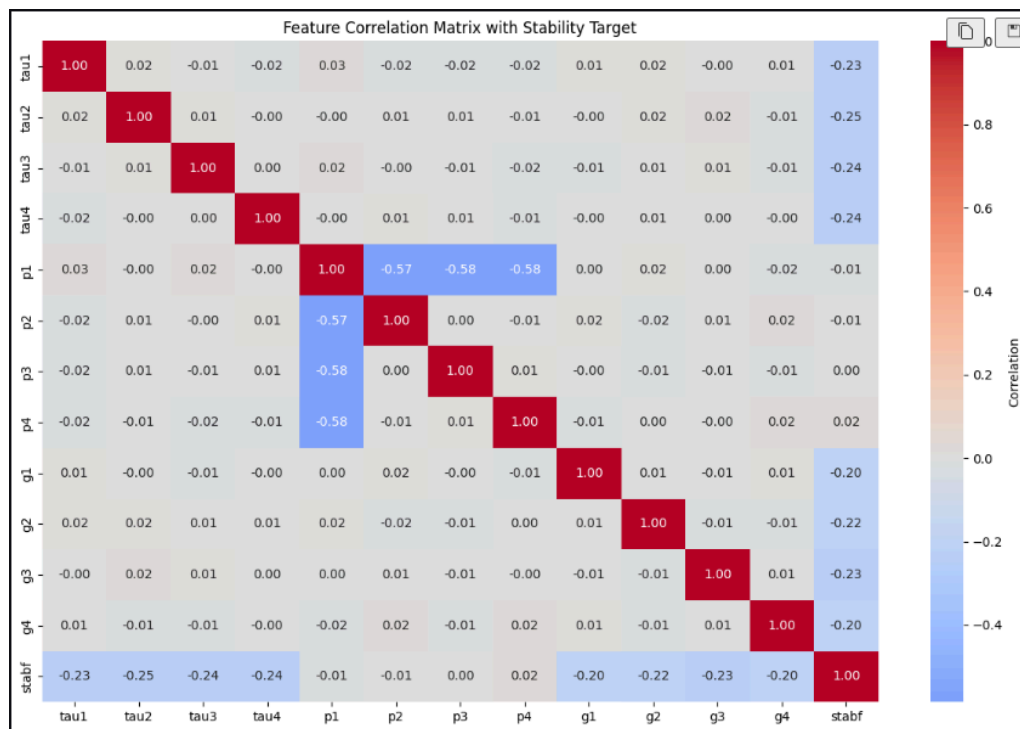
9. PRECISION-RECALL CURVE:

This graph shows the relationship (graphical relationship) between **Precision** (True positives) and **recall** (which is the true positive rate)



10. FEATURE CORRELATION MATRIX WITH STABILITY TARGET

Department of electrical/electronics



11. ALL OUTPUTS FOR EACH CELL OF CODE:

CELL 1

```
✓ Dataset validation passed

=====
DATA PREPROCESSING & CLASS BALANCE CHECK
=====

✓ Class Distribution BEFORE splitting:
Unstable (Class 0): 6388 samples (63.88%)
Stable (Class 1): 3620 samples (36.20%)
Training set size: 8000 samples
Testing set size: 2000 samples

=====
FEATURE CORRELATION WITH STABILITY
=====

Correlation with Stability (1=Stable, 0=Unstable):
p4 0.022785
p3 0.000649
p2 -0.006173
p1 -0.009938
g1 -0.197664
g4 -0.204931
g2 -0.217341
g3 -0.231774
tau1 -0.234898
tau3 -0.237492
tau4 -0.239375
tau2 -0.246280
dtype: float64
```



CELL 2

```
=====
DATA SCALING (for distance-based algorithms)
=====
StandardScaler centers features to mean=0, std=1
This is REQUIRED for Logistic Regression but NOT for Decision Trees

✓ Scaling complete with feature names preserved
Scaled training data shape: (8000, 12)
Scaled testing data shape: (2000, 12)
Features: ['tau1', 'tau2', 'tau3', 'tau4', 'p1', 'p2', 'p3', 'p4', 'g1', 'g2', 'g3', 'g4']
Sample scaled values (first row): [-0.424656  1.25667699  0.14023222]
```

CELL 3:

```
✓ Model trained successfully

--- CROSS-VALIDATION (5-Fold Stratified) ---
Cross-Val Accuracy: 0.8154 ± 0.0077
Cross-Val F1-Score: 0.7344 ± 0.0111
Individual fold accuracies: ['0.8100', '0.8087', '0.8269', '0.8087', '0.8225']

=====
TEST SET PERFORMANCE
=====
Accuracy:      81.85%
Balanced Accuracy:  79.25% (better for imbalanced)
F1-Score:      0.7364 (harmonic mean of precision & recall)

Detailed Classification Report:
      precision    recall  f1-score   support

Unstable (0)       0.84       0.89       0.86       1276
Stable (1)         0.78       0.70       0.74        724

   accuracy       0.82       2000
  macro avg       0.81       0.79       0.80       2000
weighted avg       0.82       0.82       0.82       2000

Confusion Matrix:
              Predicted
              Unstable  Stable
Actual Unstable   1130     146
Actual Stable      217     507
```

CELL 4

```
=====
FEATURE IMPORTANCE & GRID VULNERABILITY ANALYSIS
=====

Feature Importance Ranking:
(Positive = Stabilizing | Negative = Destabilizing)

tau4      | Coef: -0.9074 | DESTABILIZING | Impact: 0.9074
tau1      | Coef: -0.8910 | DESTABILIZING | Impact: 0.8910
tau2      | Coef: -0.8867 | DESTABILIZING | Impact: 0.8867
tau3      | Coef: -0.8657 | DESTABILIZING | Impact: 0.8657
g3        | Coef: -0.8605 | DESTABILIZING | Impact: 0.8605
g2        | Coef: -0.8129 | DESTABILIZING | Impact: 0.8129
g4        | Coef: -0.7489 | DESTABILIZING | Impact: 0.7489
g1        | Coef: -0.7301 | DESTABILIZING | Impact: 0.7301
p3        | Coef: -0.0610 | DESTABILIZING | Impact: 0.0610
p4        | Coef:  0.0405 | STABILIZING   | Impact: 0.0405
p2        | Coef:  0.0142 | STABILIZING   | Impact: 0.0142
p1        | Coef:  0.0034 | STABILIZING   | Impact: 0.0034

Model Intercept (bias): -1.0558

✓ Feature importance saved to: ./outputs/feature_importance_logistic.csv
```




CELL 5:

```
=====
DECISION TREE MODEL - COMPARISON & EVALUATION
=====
🔑 KEY DIFFERENCE: Decision Trees work on UNSCALED data (no scaling needed)

✓ Decision Tree trained on UNSCALED data

--- CROSS-VALIDATION (5-Fold Stratified) ---
Cross-Val Accuracy: 0.8455 ± 0.0116
Cross-Val F1-Score: 0.7960 ± 0.0146

=====
MODEL COMPARISON: LOGISTIC REGRESSION VS DECISION TREE
=====

Metric Logistic Regression Decision Tree
Accuracy          81.85%          85.85%
Balanced Accuracy  79.29%          85.50%
F1-Score          0.7364          0.8117
CV Accuracy Mean  0.8154          0.8455

✓ BEST MODEL: Decision Tree (85.85% accuracy)
```

CELL 6:

```
--- DECISION TREE FEATURE IMPORTANCE ---
Feature Importance
3   tau4    0.153021
2   tau3    0.146177
1   tau2    0.145699
0   tau1    0.128405
8   g1      0.115032
```

CELL 7:



```
0.60      0.612      0.922      0.816      0.699
0.70      0.507      0.956      0.868      0.640
0.75      0.463      0.969      0.896      0.610
0.80      0.388      0.979      0.912      0.545
0.90      0.233      0.997      0.977      0.377

✓ RECOMMENDED THRESHOLD: 0.50 (F1-Score: 0.7364)

=====
NERC EARLY WARNING SYSTEM (Threshold = 75%)の時間
=====
Interpretation: Grid must have ≥75% confidence to be declared SAFE
Otherwise: EMERGENCY ACTION TRIGGERED

Classification Report:
      precision    recall  f1-score   support

Emergency Action      0.76      0.97      0.85      1276
  Grid Safe          0.90      0.46      0.61       724

   accuracy
 macro avg      0.83      0.72      0.73      2000
weighted avg      0.81      0.79      0.76      2000

Operational Breakdown:
True Negatives (Correct Emergency): 1237 (avoiding false alarms)
False Positives (False Alarms): 39 (unnecessary activations)
False Negatives (MISSED RISKS): 389 ⚠️ CRITICAL
True Positives (Correct Safe): 335
```

CELL 8

Part 1

```
=====
PREDICTION METADATA & CONFIDENCE ANALYSIS
=====

Sample Predictions with Confidence Metrics:
      Actual_Stability  Predicted_Stability  Prediction_Confidence  Probability_Stable  Correct_Prediction
0                0                0                0.926760                0.073240                1
1                0                0                0.976694                0.023306                1
2                0                0                0.968499                0.031501                1
3                1                1                0.730231                0.730231                1
4                0                0                0.501402                0.498598                1
5                1                1                0.843976                0.843976                1
6                1                1                0.511465                0.511465                1
7                0                0                0.992617                0.007383                1
8                0                0                0.968502                0.031498                1
9                1                1                0.833186                0.833186                1

Confidence Distribution:
Low Confidence (<60%): 236 predictions
Medium Confidence: 549 predictions
High Confidence (≥80%): 1215 predictions

✓ Prediction metadata saved to: ./outputs/prediction\_metadata.csv

=====
MODEL PERSISTENCE - SAVING FOR PRODUCTION
=====

Validating models before persistence...
```

Part 2

```
Medium Confidence:      549 predictions
High Confidence (≥80%): 1215 predictions

✓ Prediction metadata saved to: ./outputs/prediction\_metadata.csv

=====
MODEL PERSISTENCE - SAVING FOR PRODUCTION
=====

Validating models before persistence...
✓ Model validation passed
✓ Model saved to: ./models/voltage\_stability\_model.pkl
✓ Scaler saved to: ./models/voltage\_scaler.pkl
✓ Config saved to: ./models/config.pkl
✓ Metrics saved to: ./outputs/model\_metrics.pkl

To use the saved model in production:
loaded_model = joblib.load('./models/voltage_stability_model.pkl')
loaded_scaler = joblib.load('./models/voltage_scaler.pkl')
loaded_config = joblib.load('./models/config.pkl')
X_new_scaled = loaded_scaler.transform(X_new)
predictions = loaded_model.predict(X_new_scaled)
probabilities = loaded_model.predict_proba(X_new_scaled)

=====
✓ ANALYSIS COMPLETE
=====

Outputs saved to: ./outputs/
Models saved to: ./models/
```

And that's a wrap!